DESARROLLO DE SISTEMAS

Modelado UML aplicado al desarrollo de software

Apunte teórico integral y desarrollado - 6.º año
Material de estudio, trabajo en clase y preparación para rendir la materia

---

Recorrido de la unidad

Relevamiento → Requerimientos → Modelado UML → Construcción de diagramas → Casos de Uso → Diagrama de Clases → Trazabilidad → Diseño de Interfaz

Este apunte está pensado para poder estudiarse de manera autónoma. Por eso no se limita a definir conceptos: desarrolla para qué sirven, cómo se construyen, cómo se interpretan, cómo se relacionan y cómo se aplican sobre una misma problemática.

1. Introducción: desarrollar no es solamente programar

Cuando se habla de desarrollo de sistemas es común pensar inmediatamente en código, lenguajes de programación, bases de datos o pantallas. Sin embargo, programar es solamente una parte del proceso. Antes de construir una solución es necesario comprender qué problema existe, quiénes están involucrados, qué necesitan, qué reglas deben respetarse y qué comportamiento se espera del sistema.

Si se comienza a programar sin haber comprendido el problema, es posible construir técnicamente algo que funcione pero que no resuelva la necesidad real. Por eso el desarrollo se apoya en actividades previas de relevamiento, análisis y diseño.

De manera general podemos representar el recorrido como: Relevamiento → Requerimientos → Análisis → Diseño → Desarrollo → Pruebas → Implementación. Estas etapas no deben entenderse como compartimentos aislados. La información obtenida en una etapa condiciona las decisiones de las siguientes.

> Ejemplo: “El sistema deberá permitir que los estudiantes consulten las cursadas disponibles y realicen una inscripción”. Esta frase todavía no indica quién administra las cursadas, qué datos posee cada una, cómo se controla el cupo, qué sucede si no hay disponibilidad o cómo se registra la inscripción. El análisis y el modelado permiten transformar una necesidad general en una solución comprensible y construible.

2. ¿Qué significa modelar un sistema?

Modelar significa construir una representación simplificada de una realidad con el objetivo de comprenderla, analizarla o diseñarla. Un plano arquitectónico no es el edificio terminado, pero permite observar su distribución, dimensiones y relaciones antes de construirlo. En software sucede algo semejante.

Un modelo no intenta copiar absolutamente todos los detalles de la realidad. Selecciona los aspectos que resultan importantes para el propósito del análisis. Por eso pueden existir distintos modelos del mismo sistema: uno puede concentrarse en las funcionalidades que utiliza el usuario y otro en la estructura interna necesaria para sostenerlas.

El modelado resulta útil porque permite discutir una solución antes de invertir tiempo en programarla. También ayuda a descubrir omisiones, contradicciones o decisiones poco claras. Cuando varias personas trabajan sobre un proyecto, un modelo común facilita la comunicación y reduce interpretaciones diferentes.

- Comprender y comunicar el problema.
- Organizar la solución antes de programar.
- Representar diferentes vistas del mismo sistema.
- Detectar errores u omisiones tempranamente.
- Documentar decisiones de análisis y diseño.
- Relacionar requerimientos con elementos concretos de la solución.

3. UML: Lenguaje Unificado de Modelado

UML (Unified Modeling Language o Lenguaje Unificado de Modelado) es un lenguaje gráfico estandarizado utilizado para visualizar, especificar, diseñar y documentar sistemas, especialmente sistemas de software.

La palabra lenguaje es importante: UML establece una forma compartida de representar determinados conceptos. Así como un lenguaje escrito posee reglas y símbolos, UML utiliza figuras, líneas, flechas y relaciones con significados definidos.

UML no es un lenguaje de programación. Un diagrama UML no se ejecuta como un programa. Tampoco es una aplicación determinada: draw.io, Lucidchart, Visual Paradigm u otras herramientas pueden utilizarse para dibujar UML, pero ninguna de ellas es UML en sí misma.

UML tampoco es una metodología completa de desarrollo. No indica por sí solo cómo organizar todo un proyecto. Es una herramienta de modelado que puede utilizarse dentro de diferentes formas de trabajo.

4. ¿Por qué UML posee diferentes diagramas?

Un sistema puede analizarse desde distintos puntos de vista. Si queremos saber qué puede hacer un estudiante, necesitamos observar el comportamiento que el sistema ofrece al usuario. Si queremos saber qué información existe y cómo se relaciona, necesitamos observar su estructura.

Por esa razón UML posee distintos diagramas. No se utiliza un único dibujo para representar absolutamente todo. Cada tipo de diagrama responde mejor a determinadas preguntas.

4.1 Diagramas estructurales

Representan principalmente cómo está constituido el sistema. Permiten observar elementos relativamente estables de la solución y las relaciones entre ellos. Entre los ejemplos se encuentran los diagramas de clases, objetos, componentes y despliegue.

4.2 Diagramas de comportamiento

Representan principalmente qué hace el sistema, cómo responde o cómo se desarrolla un proceso. Entre ellos se encuentran los diagramas de casos de uso, actividades y estados. Los diagramas de interacción, como secuencia, también permiten estudiar el comportamiento.

> En esta unidad se profundizan dos vistas complementarias: Casos de Uso, para observar funcionalidades y actores, y Diagrama de Clases, para observar la estructura del sistema.

5. Relación entre relevamiento, requerimientos y UML

Los diagramas no deberían aparecer como dibujos aislados. Su contenido debe surgir del problema previamente analizado. Durante el relevamiento se obtiene información de usuarios, documentos, procesos y reglas. Esa información se analiza y se expresa mediante requerimientos.

Un requerimiento funcional describe una capacidad que el sistema debe ofrecer. Por ejemplo: RF01: El sistema deberá permitir al estudiante buscar las cursadas disponibles. A partir de ese requerimiento podemos identificar un actor, Estudiante, y una funcionalidad, Buscar Cursada.

Otro requerimiento podría ser: RF02: El sistema deberá permitir al estudiante confirmar una inscripción cuando exista disponibilidad. Esto aporta información para modelar el comportamiento y también permite comenzar a reconocer conceptos estructurales como Estudiante, Inscripción, Cursada o Cupo.

> Continuidad: necesidad detectada → requerimiento → representación UML → diseño → implementación. El diagrama no reemplaza al requerimiento: lo complementa y permite observarlo desde otra perspectiva.

6. Cómo se realiza físicamente un diagrama UML

Un alumno puede comprender perfectamente UML sin utilizar una computadora. Los diagramas pueden realizarse a mano o mediante herramientas digitales. Lo importante es que la representación respete la notación y que el contenido sea coherente con el problema.

6.1 Realización a mano

Para aprender, practicar o rendir una evaluación, puede utilizarse hoja, lápiz y regla. El procedimiento intelectual es exactamente el mismo que en una herramienta digital: primero se analiza el problema, después se identifican los elementos y finalmente se dibujan.

La ventaja del papel es que permite concentrarse en el razonamiento sin depender de una aplicación. En una evaluación, un diagrama a mano es válido siempre que resulte legible, utilice los símbolos correctos y exprese las relaciones de manera comprensible.

6.2 Realización digital

Una herramienta digital facilita mover elementos, corregir nombres, alinear figuras, modificar conectores y exportar el resultado. Para la materia puede utilizarse draw.io / diagrams.net, que incluye bibliotecas con elementos UML.

La herramienta no decide qué actor existe, qué caso de uso corresponde ni qué clases necesita el sistema. Es solamente el medio de dibujo. El análisis continúa siendo responsabilidad de quien modela.

6.3 Secuencia práctica para crear un gráfico

- Leer el problema completo. No comenzar a dibujar con la primera palabra que aparece.
- Determinar qué se quiere representar. Si interesa funcionalidad y usuarios, Casos de Uso. Si interesa estructura, Diagrama de Clases.
- Identificar los elementos relevantes. Actores y acciones, o clases, atributos y relaciones.
- Realizar un primer borrador. El primer gráfico no tiene que ser definitivo.
- Agregar relaciones. Las líneas y flechas deben expresar relaciones reales, no decorar el dibujo.
- Comparar el gráfico con los requerimientos. Cada elemento debe poder justificarse.
- Corregir y ordenar. Recién al final se mejora la presentación visual.

6.4 Procedimiento básico en draw.io / diagrams.net

Al crear un diagrama nuevo se puede abrir la opción de bibliotecas o formas adicionales y habilitar las categorías relacionadas con UML. Luego se arrastran los elementos al área de trabajo. El texto de cada figura puede editarse y las relaciones se realizan con conectores.

Para Casos de Uso se utilizan principalmente actores, óvalos de casos de uso, el límite del sistema y conectores. Para Diagrama de Clases se utilizan rectángulos de clase divididos en sectores y conectores para asociaciones, herencia y otras relaciones.

> Regla práctica para el alumno: primero pienso y analizo → después elijo el tipo de diagrama → luego lo dibujo a mano o con una herramienta. El programa no reemplaza el razonamiento.

7. Diagrama de Casos de Uso: finalidad

El Diagrama de Casos de Uso pertenece a los diagramas de comportamiento. Su finalidad principal es mostrar qué servicios o funcionalidades ofrece el sistema y qué actores externos participan en ellos.

Este diagrama resulta especialmente útil durante el análisis porque permite conversar sobre lo que el sistema debe permitir hacer sin entrar todavía en detalles de programación. Un usuario puede comprender con relativa facilidad un gráfico que indica que un Estudiante puede Buscar Cursada o Confirmar Inscripción.

La pregunta central es: ¿qué objetivo puede alcanzar cada actor utilizando el sistema? Por eso el diagrama se concentra en objetivos y funcionalidades, no en botones, campos de texto, tablas de base de datos o algoritmos internos.

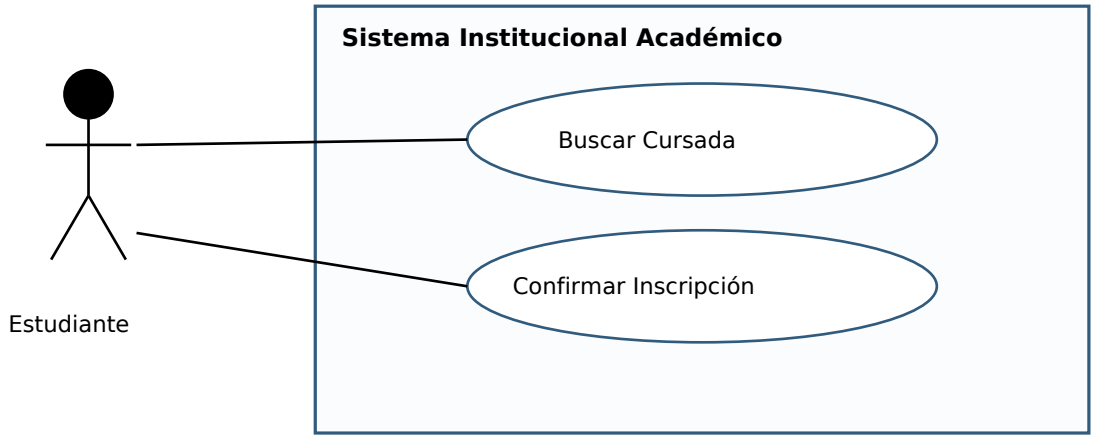


Figura 1. Actor, límite del sistema, casos de uso y asociaciones.

8. Actor: qué representa y cómo identificarlo

Un actor representa un rol externo que interactúa con el sistema. Puede ser una persona, un grupo de personas, otra aplicación, un dispositivo o un servicio externo.

La palabra rol evita confundir el modelo con personas concretas. En un sistema escolar no interesa representar a “Juan Pérez” como individuo, sino el papel que cumple: Estudiante, Docente, Preceptor o Administrador.

Para identificar actores conviene preguntar: ¿quién inicia una acción?, ¿quién recibe información del sistema?, ¿qué otro sistema intercambia información?, ¿quién administra o controla datos?

Un mismo individuo podría desempeñar más de un rol. Por ejemplo, una persona podría ser Docente y también Administrador. En el diagrama se representan los roles porque poseen responsabilidades diferentes frente al sistema.

9. Caso de uso: qué representa

Un caso de uso representa una funcionalidad que permite a un actor alcanzar un objetivo. Se dibuja mediante un óvalo y su nombre debería expresar una acción comprensible.

Por eso es habitual utilizar la forma verbo + objeto: Buscar Cursada, Confirmar Inscripción, Consultar Inscripciones, Crear Cursada, Generar Informe.

Un nombre como “Inscripción” es ambiguo porque no indica qué sucede. ¿Se crea, se consulta, se cancela o se confirma? En cambio “Confirmar Inscripción” expresa con mayor claridad el objetivo.

El nivel de detalle también debe ser razonable. “Presionar botón Confirmar” es demasiado cercano a la interfaz. “Gestionar todo el sistema” es demasiado general. El caso de uso debe representar un objetivo significativo para el actor.

10. Límite del sistema

El límite del sistema suele representarse mediante un rectángulo que contiene los casos de uso. Su función es indicar qué comportamiento pertenece al sistema que estamos analizando y qué elementos se encuentran fuera de él.

Los actores se ubican fuera porque representan entidades externas que interactúan con la solución. Esto no significa que un Estudiante sea poco importante: significa que el actor no es una funcionalidad interna del software.

Definir correctamente el límite ayuda a evitar confusiones de alcance. Si un servicio de correo electrónico es externo, puede representarse como actor o sistema externo, mientras que la funcionalidad que solicita el envío pertenece al sistema académico.

11. Asociación entre actor y caso de uso

La asociación se representa mediante una línea que conecta un actor con un caso de uso. Indica que existe interacción entre ambos.

Si Estudiante está asociado con Buscar Cursada, significa que el actor participa en esa funcionalidad. La línea no indica por sí sola el orden de los pasos ni cómo se implementa técnicamente.

Un actor puede asociarse con varios casos de uso. Un mismo caso también puede relacionarse con más de un actor si ambos participan realmente de esa funcionalidad.

12. Relación <<include>>

La relación <<include>> se utiliza cuando un caso de uso incorpora obligatoriamente el comportamiento de otro caso o cuando un comportamiento común se reutiliza desde diferentes casos.

Supongamos que para Confirmar Inscripción siempre es necesario Verificar Cupo Disponible. La verificación no es opcional: forma parte necesaria del proceso. Entonces Confirmar Inscripción incluye Verificar Cupo Disponible.

Gráficamente se utiliza una línea discontinua con flecha abierta dirigida hacia el caso incluido y la etiqueta <<include>>. La dirección es importante: la flecha apunta hacia la funcionalidad que está siendo incluida.

Separar un comportamiento mediante include puede mejorar la claridad y evitar repetirlo cuando varias funcionalidades necesitan exactamente la misma acción.

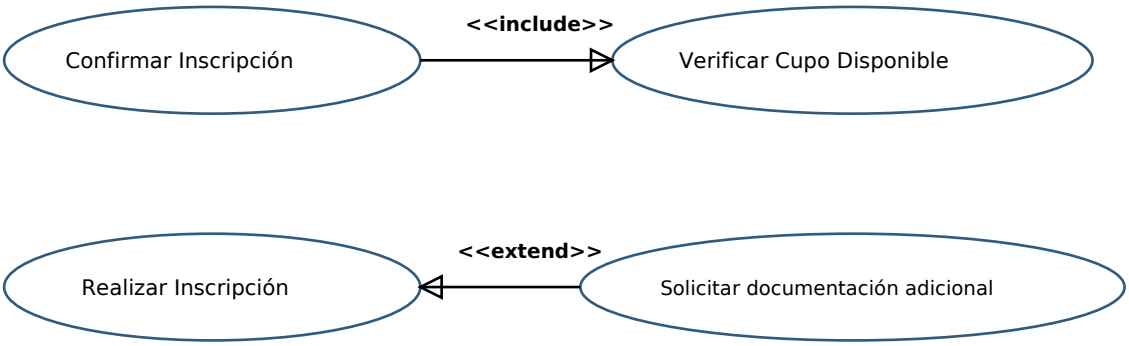


Figura 2. Diferencia gráfica entre <> y <>.

13. Relación <>

La relación <<extend>> representa un comportamiento adicional que ocurre solamente bajo determinadas condiciones. El caso principal puede completarse sin que la extensión ocurra en todos los casos.

Por ejemplo, si durante una inscripción solamente algunos estudiantes deben presentar documentación adicional, Solicitar Documentación Adicional podría modelarse como una extensión de Realizar Inscripción, siempre que esa regla corresponda realmente al sistema.

La diferencia conceptual es fundamental: include expresa comportamiento necesario o reutilizado; extend expresa comportamiento condicionado. No se elige una relación porque “queda mejor” en el gráfico.

Antes de usar extend conviene poder responder: ¿cuál es la condición que hace aparecer este comportamiento? Si no existe una condición clara, probablemente la relación esté mal elegida.

14. Generalización de actores

La generalización permite representar que varios actores especializados comparten características de un actor más general. Por ejemplo, Estudiante, Docente y Administrador podrían ser especializaciones de Usuario.

Esta relación debe utilizarse cuando aporta información real. No es obligatorio crear un actor general solamente porque varios actores sean personas. El modelo debe mantenerse tan simple como sea posible, pero tan completo como sea necesario.

15. Construcción paso a paso de un Diagrama de Casos de Uso

Supongamos la siguiente problemática: “El sistema académico permite que un estudiante busque cursadas disponibles y confirme su inscripción. Antes de confirmar, el sistema debe verificar que exista cupo. Los docentes pueden consultar las inscripciones y gestionar instancias de evaluación. El administrador puede gestionar cursadas y generar informes.”

Paso 1: identificar actores

Aparecen tres roles humanos claramente diferenciados: Estudiante, Docente y Administrador. No se inventan otros actores si el texto no los necesita.

Paso 2: identificar objetivos

Del Estudiante surgen Buscar Cursada y Confirmar Inscripción. Del Docente surgen Consultar Inscripciones y Gestionar Instancias de Evaluación. Del Administrador surgen Gestionar Cursada y Generar Informes.

Paso 3: analizar comportamientos internos reutilizables

La frase “antes de confirmar, el sistema debe verificar que exista cupo” permite identificar Verificar Cupo Disponible como comportamiento necesario de Confirmar Inscripción.

Paso 4: dibujar y relacionar

Se coloca el sistema dentro de un límite, los actores fuera, los casos dentro y las asociaciones correspondientes. Luego se agrega la relación include entre Confirmar Inscripción y Verificar Cupo Disponible.

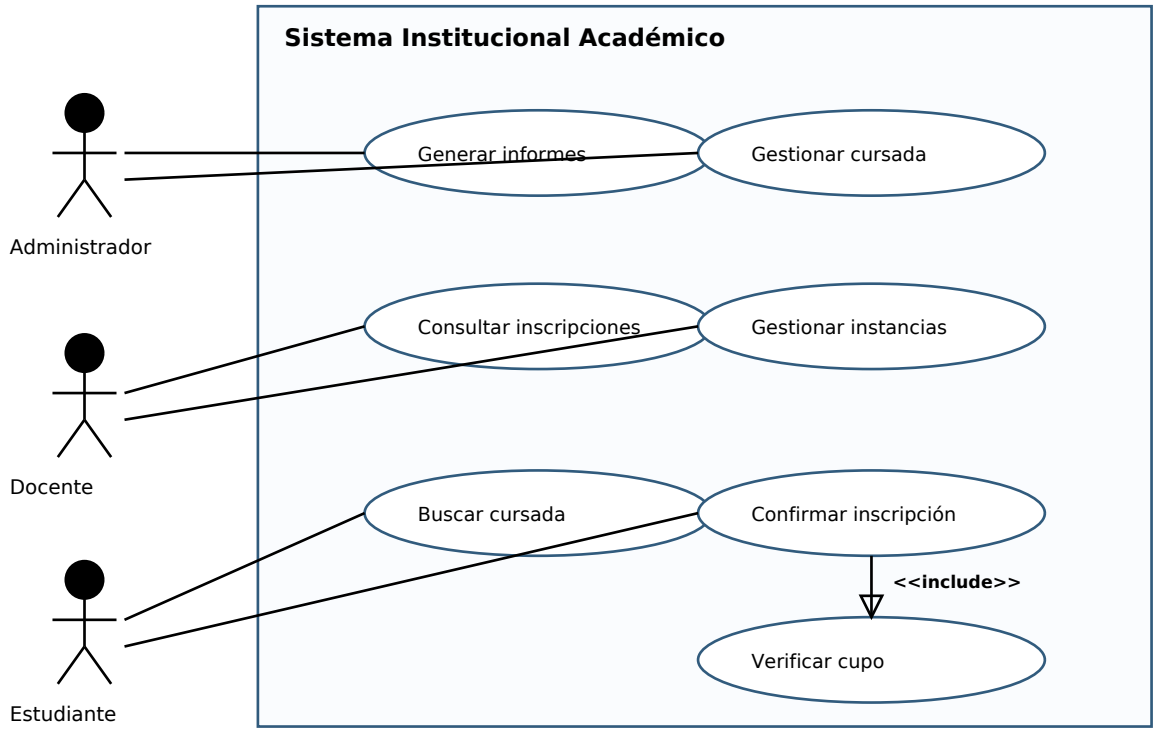


Figura 3. Ejemplo integrado de Casos de Uso sobre el Sistema Institucional Académico.

Paso 5: revisar

Se compara el diagrama con la problemática. Si aparece una funcionalidad que no puede justificarse en el texto o falta una funcionalidad solicitada, el modelo debe corregirse.

16. Errores frecuentes en Casos de Uso

- Confundir actor con persona concreta. El modelo representa roles.
- Nombrar casos como sustantivos. Conviene expresar la acción.
- Dibujar pantallas o botones. Eso pertenece al diseño de interfaz.
- Utilizar include o extend sin comprender su significado.
- Agregar casos para que el gráfico parezca más completo. Todo elemento debe justificarse.
- Representar pasos demasiado pequeños. Un caso de uso debe expresar un objetivo significativo.
- Olvidar el alcance. El límite del sistema ayuda a distinguir lo interno de lo externo.

17. Del comportamiento a la estructura

El Diagrama de Casos de Uso nos permite comprender qué debe poder hacer el sistema. Sin embargo, todavía no muestra qué elementos internos necesita la solución para sostener esas funcionalidades.

Si el sistema debe Buscar Cursada, aparece el concepto Cursada. Si debe Confirmar Inscripción, aparecen conceptos como Estudiante e Inscripción. Si debe controlar cupos de instancias de evaluación, aparecen Instancia de Evaluación y Cupo.

Esta observación no significa que cada palabra se convierta automáticamente en una clase. Significa que los requerimientos y casos de uso proporcionan información para descubrir conceptos relevantes del dominio.

18. Diagrama de Clases: finalidad

El Diagrama de Clases es un diagrama estructural. Representa la estructura estática del sistema mediante clases, atributos, operaciones y relaciones.

La palabra estática no significa que el sistema no cambie. Significa que el diagrama muestra principalmente qué tipos de elementos existen y cómo están relacionados, en lugar de representar una secuencia temporal de acciones.

Este diagrama es fundamental porque permite pasar de una descripción funcional a una representación más cercana al diseño orientado a objetos.

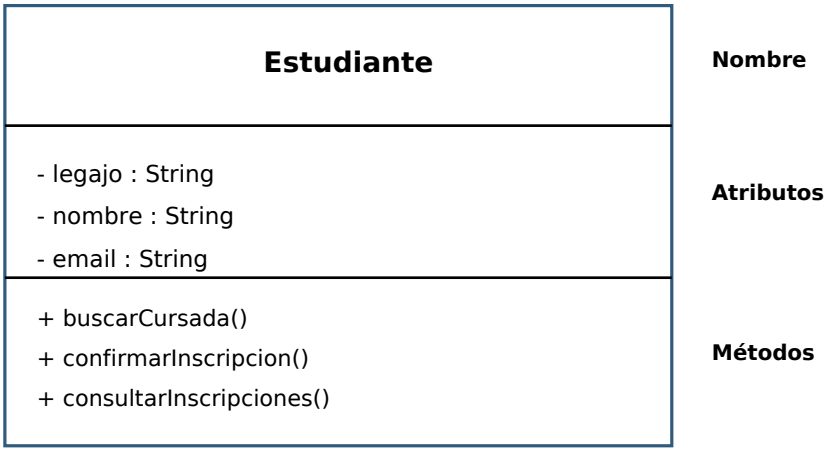


Figura 4. Anatomía de una clase UML: nombre, atributos y métodos.

19. Clase: concepto

Una clase representa un conjunto de objetos que comparten características y comportamientos. Puede entenderse como una definición general a partir de la cual existirán objetos concretos.

Por ejemplo, Estudiante puede ser una clase. Un estudiante particular, con un legajo y un nombre determinados, sería una instancia u objeto de esa clase.

En UML una clase suele dibujarse como un rectángulo dividido en tres sectores: nombre de la clase, atributos y métodos u operaciones.

20. Atributos

Los atributos representan datos o características que los objetos de una clase necesitan mantener. En Estudiante podrían existir legajo, nombre, apellido o email. En Cursada podrían existir identificador, nombre, comisión o turno.

UML permite indicar el tipo de dato utilizando la forma nombreAtributo : Tipo. Ejemplos: idCursada : int, nombre : String, fecha : Date, activo : boolean.

Un atributo debe pertenecer conceptualmente a la clase. Si un dato describe a la Cursada, no debería colocarse arbitrariamente en Estudiante.

21. Métodos u operaciones

Los métodos u operaciones representan comportamientos o responsabilidades asociados a una clase. Por ejemplo, Estudiante podría participar en buscarCursada() o confirmarInscripcion(), mientras que InstanciaEvaluacion podría ofrecer verificarCupo().

Una operación puede recibir parámetros. Por ejemplo: buscarCursada(nombre : String). Esto indica que para realizar la búsqueda se proporciona un dato de tipo String.

En el diagrama no es necesario escribir el código del método. Se expresa la responsabilidad que la clase tendrá dentro de la solución.

22. Visibilidad

UML permite indicar qué tan accesible es un atributo o método. Los símbolos más conocidos son + para público, - para privado y # para protegido.

Por ejemplo, -email : String indica un atributo privado y +buscarCursada() una operación pública. La visibilidad se relaciona con la forma en que los objetos protegen y exponen su información.

23. Encapsulamiento

El encapsulamiento es un principio de la orientación a objetos que consiste en agrupar datos y comportamientos relacionados dentro de una clase y controlar el acceso a su estado interno.

La idea no es ocultar información por capricho, sino evitar que cualquier parte del sistema modifique datos sin control. Una clase puede proporcionar operaciones que validen o administren esos cambios.

El encapsulamiento favorece el mantenimiento porque reduce dependencias y permite modificar la implementación interna de una clase sin afectar necesariamente a todo el sistema.

24. Asociación entre clases

Una asociación expresa que dos clases mantienen un vínculo relevante dentro del dominio. Por ejemplo, Estudiante puede estar relacionado con Inscripción.

La asociación no significa simplemente que dos nombres aparecen cerca. Debe existir una relación real que el sistema necesite conocer o administrar.

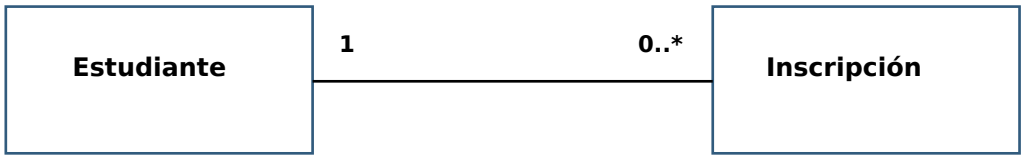
En un modelo más detallado, una asociación puede tener nombre, dirección o roles, pero en esta etapa resulta suficiente comprender qué clases se vinculan y con qué multiplicidad.

25. Multiplicidad

La multiplicidad indica cuántas instancias de una clase pueden relacionarse con una instancia de otra. Es una forma de expresar reglas del sistema dentro del modelo.

1 significa exactamente una instancia. 0..1 significa ninguna o una. 0..* significa ninguna o muchas. 1..* significa una o muchas.

Si un Estudiante puede tener varias inscripciones, una relación posible podría indicar 1 del lado de Estudiante y 0..* del lado de Inscripción. Esto debe interpretarse según la dirección de lectura y las reglas concretas del sistema.



Una instancia de Estudiante puede relacionarse con cero o muchas Inscripciones.

Figura 5. Ejemplo de asociación con multiplicidad.

Las multiplicidades no deben adivinarse. Deben surgir del relevamiento y de las reglas de negocio. Si no sabemos cuántas inscripciones puede tener un estudiante, debemos volver a los requerimientos o consultar al usuario.

26. Herencia o generalización entre clases

La herencia permite representar una relación de especialización. Una clase general contiene características comunes y las clases especializadas las heredan, pudiendo agregar otras propias.

Por ejemplo, Usuario puede contener idUsuario, nombre, apellido y email. Estudiante, Docente y Administrador son tipos específicos de Usuario y pueden agregar legajos u operaciones particulares.

La herencia evita repetir información común y, principalmente, expresa una relación conceptual: un Estudiante es un Usuario; un Docente es un Usuario.

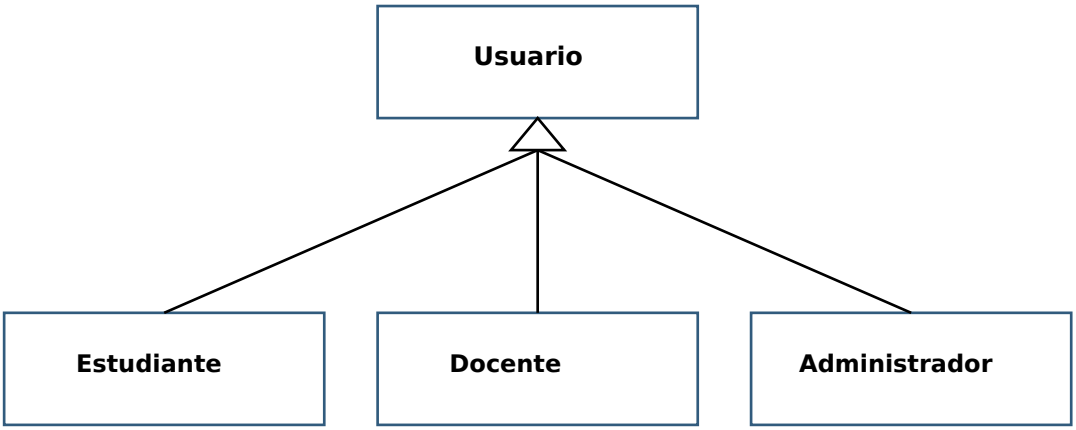


Figura 6. Generalización: Estudiante, Docente y Administrador especializan a Usuario.

27. Agregación y composición

Además de la asociación y la herencia, UML permite expresar relaciones todo-parte.

La agregación representa una relación en la que un elemento contiene o agrupa otros, pero las partes pueden existir independientemente. Se representa con un rombo vacío.

La composición representa una relación más fuerte: la parte depende del todo para su existencia dentro del modelo. Se representa con un rombo relleno.

Estas relaciones no deben utilizarse solamente para hacer el diagrama más sofisticado. Si no existe una regla clara que justifique una relación todo-parte, una asociación simple suele ser más correcta.

28. Cómo identificar clases a partir del problema

Una técnica inicial consiste en leer los requerimientos y detectar conceptos relevantes del dominio. Los sustantivos pueden servir como candidatos, pero no todos se transforman automáticamente en clases.

Para evaluar un candidato conviene preguntar si el sistema necesita almacenar información sobre él, si posee responsabilidades propias, si participa de relaciones importantes y si representa un concepto significativo del problema.

En el Sistema Institucional Académico, Usuario, Estudiante, Docente, Administrador, Cursada, Inscripción e Instancia de Evaluación son conceptos que pueden justificarse a partir de las funcionalidades planteadas.

> Advertencia: identificar sustantivos es una ayuda inicial, no una regla automática. “Correo”, “botón”, “informe” o “pantalla” no necesariamente deben convertirse en clases del dominio.

29. No todo Caso de Uso se convierte en una clase

Este es uno de los errores más frecuentes al comenzar. Un caso de uso representa una funcionalidad; una clase representa un elemento estructural.

Buscar Cursada no obliga a crear una clase BuscarCursada. Confirmar Inscripción tampoco obliga a crear ConfirmarInscripcion como clase. Esas funcionalidades pueden ser realizadas mediante la colaboración de varias clases.

La pregunta correcta no es “¿cómo convierto cada óvalo en un rectángulo?”, sino “¿qué objetos y responsabilidades necesita el sistema para poder realizar estas funciones?”

30. Construcción paso a paso de un Diagrama de Clases

Tomemos la misma problemática utilizada en Casos de Uso. El sistema administra usuarios con distintos roles, cursadas, instancias de evaluación e inscripciones.

Paso 1: identificar conceptos principales

Usuario aparece como concepto general. Estudiante, Docente y Administrador poseen características específicas. También aparecen Cursada, Instancia de Evaluación e Inscripción.

Paso 2: asignar atributos

Usuario puede contener identificador, nombre y email. Estudiante y Docente pueden agregar sus respectivos legajos. Cursada puede contener identificador, nombre y turno. Instancia de Evaluación puede contener fecha y cupo. Inscripción puede contener identificador, fecha y estado.

Paso 3: asignar responsabilidades

No se colocan métodos al azar. Las operaciones deben relacionarse con las responsabilidades del objeto. Por ejemplo, InstanciaEvaluacion puede participar en verificarCupo(), mientras que Inscripción puede confirmar() o cambiarEstado(), según el diseño adoptado.

Paso 4: establecer relaciones

Estudiante se relaciona con Inscripción. Cursada se relaciona con las instancias de evaluación que contiene o administra según las reglas definidas. Usuario se generaliza en Estudiante, Docente y Administrador.

Paso 5: definir multiplicidades

Se analiza cuántos objetos pueden participar en cada relación. Esta información debe provenir de las reglas del sistema.

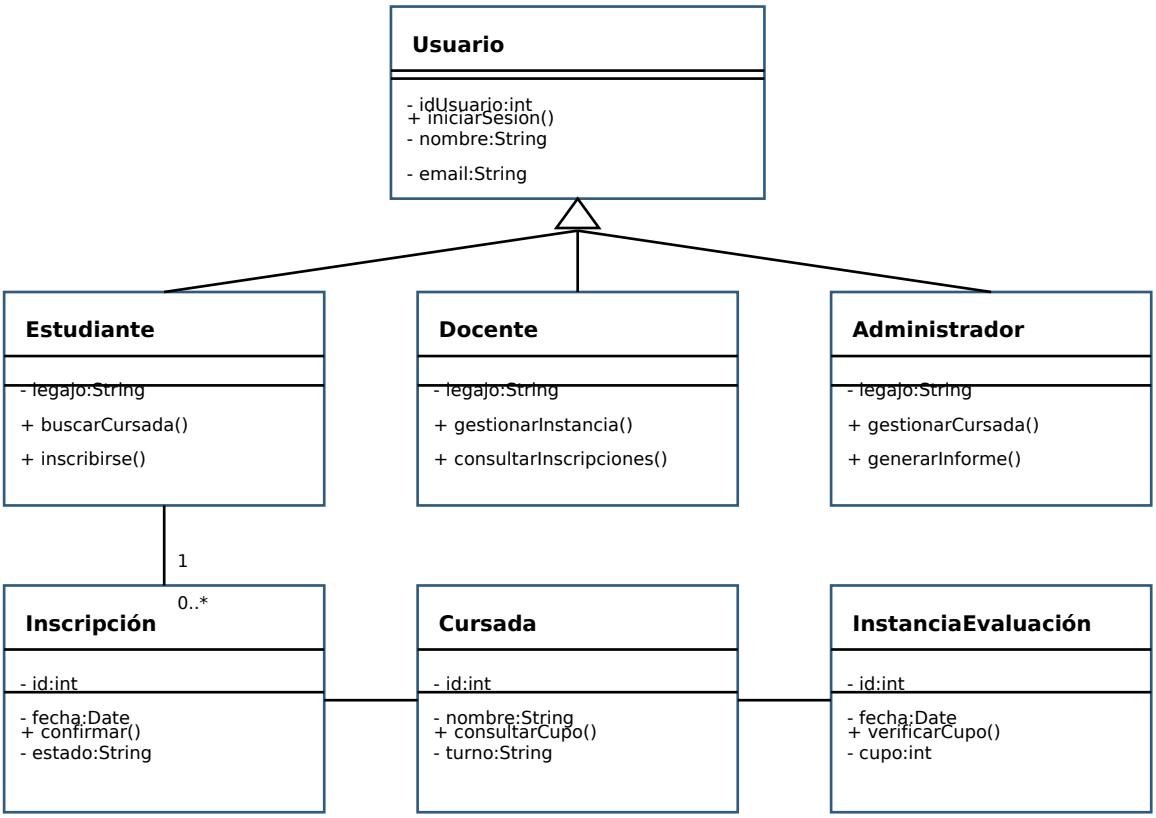


Figura 7. Modelo de clases de referencia para la misma problemática académica.

El gráfico anterior funciona como modelo didáctico. En un proyecto real, algunas relaciones o responsabilidades podrían modificarse si los requerimientos establecen reglas diferentes. El modelo debe adaptarse al problema, no al revés.

31. Coherencia entre Requerimientos, Casos de Uso y Clases

Un sistema bien analizado mantiene continuidad entre lo que se pidió y lo que se diseñó. Si un requerimiento importante no aparece reflejado en ningún modelo, existe riesgo de olvidarlo durante el desarrollo.

Supongamos: RF02: El sistema deberá permitir al estudiante confirmar una inscripción. En Casos de Uso aparece Confirmar Inscripción asociado con Estudiante. En el modelo estructural aparecen Estudiante e Inscripción, además de las clases necesarias para aplicar las reglas de disponibilidad.

No se busca que todos los modelos tengan exactamente las mismas palabras, sino que exista una relación justificable entre ellos.

32. Trazabilidad

La trazabilidad consiste en poder seguir un requerimiento a través de las diferentes etapas del proyecto. Permite conocer de dónde surgió una función, dónde fue modelada, cómo se implementó y cómo se verificará.

Una cadena sencilla puede ser: Necesidad → RF02 → CU02 Confirmar Inscripción → clases relacionadas → interfaz de inscripción → implementación → prueba.

La trazabilidad es especialmente útil cuando aparece un cambio. Si se modifica la regla de cupos, podemos identificar qué requerimiento cambia, qué caso de uso se ve afectado, qué clases participan y qué pruebas deben revisarse.

33. Diferencia entre Caso de Uso y Diagrama de Clases

Aspecto	Casos de Uso	Diagrama de Clases
Qué observa	Funcionalidad y comportamiento externo.	Estructura del sistema.
Pregunta principal	¿Qué hace el sistema y quién lo utiliza?	¿Qué elementos existen y cómo se relacionan?
Elementos	Actores, casos de uso, asociaciones, <<include>> y <<extend>>.	Clases, atributos, métodos, asociaciones, multiplicidad y herencia.
Ejemplo	Estudiante → Confirmar Inscripción	Estudiante - Inscripción - Cursada
Momento de uso	Análisis funcional.	Análisis y diseño estructural.

Los dos diagramas pueden referirse al mismo sistema, pero no deben confundirse. Un caso de uso no es una clase y una clase no representa automáticamente una función completa del usuario.

34. Ejemplo completo: del texto al modelo

Problemática: “Una institución necesita un sistema que permita a estudiantes consultar cursadas y registrarse en instancias de evaluación. Los docentes deben crear y administrar instancias, consultar inscriptos y recibir información sobre las inscripciones. Los administradores gestionan cursadas y generan informes. Antes de registrar una inscripción debe verificarse que exista cupo.”

34.1 Requerimientos que pueden derivarse

- El sistema deberá permitir al estudiante buscar cursadas.
- El sistema deberá permitir al estudiante confirmar una inscripción.
- El sistema deberá verificar disponibilidad antes de registrar la inscripción.
- El sistema deberá permitir al docente gestionar instancias de evaluación.
- El sistema deberá permitir al docente consultar inscripciones.
- El sistema deberá permitir al administrador gestionar cursadas y generar informes.

34.2 Casos de Uso

Los requerimientos anteriores permiten identificar los actores Estudiante, Docente y Administrador, junto con sus objetivos. La verificación de cupo puede modelarse como comportamiento incluido si siempre debe ejecutarse durante la confirmación.

34.3 Clases

El análisis estructural permite identificar Usuario y sus especializaciones, además de Cursada, Instancia de Evaluación e Inscripción. Luego se asignan datos y responsabilidades de acuerdo con los requerimientos.

34.4 Qué aprendemos del ejemplo

El modelo no nació de memorizar símbolos. Nació de leer una problemática, convertirla en requerimientos y utilizar UML para representar diferentes vistas de la solución. Ese es el razonamiento que debe poder reproducir el alumno.

35. ¿Cuándo puede considerarse correcto un modelo?

Un modelo no es correcto simplemente porque tenga muchas figuras o porque esté prolijo. Debe ser coherente con el problema y utilizar adecuadamente la notación.

35.1 Revisión de Casos de Uso

- ¿Los actores representan roles reales?
- ¿Los casos expresan objetivos significativos?
- ¿Los nombres indican acciones?
- ¿Las asociaciones corresponden a interacciones reales?
- ¿Los include son realmente necesarios?
- ¿Los extend poseen una condición clara?
- ¿El conjunto cubre los requerimientos funcionales trabajados?

35.2 Revisión de Clases

- ¿Cada clase representa un concepto relevante?
- ¿Los atributos pertenecen conceptualmente a la clase?
- ¿Los métodos representan responsabilidades razonables?
- ¿Las relaciones pueden justificarse?
- ¿Las multiplicidades surgen de reglas reales?
- ¿La herencia representa una relación 'es un'?
- ¿El modelo permite sostener las funcionalidades planteadas?

36. UML como puente hacia Diseño de Interfaz

Después de comprender qué debe hacer el sistema y qué elementos participan, podemos comenzar a diseñar la forma en que el usuario interactuará con esas funcionalidades.

Si el caso de uso es Buscar Cursada, una interfaz puede necesitar un mecanismo de búsqueda, filtros, listado de resultados y acceso al detalle. Si el caso es Confirmar Inscripción, la interfaz puede necesitar mostrar información de la instancia, disponibilidad, una acción de confirmación y un mensaje de resultado.

Sin embargo, el caso de uso no es la pantalla. El caso de uso expresa el objetivo. La interfaz es uno de los medios concretos que se diseñan para que el usuario pueda alcanzar ese objetivo.

Esta diferencia evita diseñar pantallas sin saber qué necesidad resuelven. Primero se comprende el problema y la función; después se diseña la interacción.

37. Recorrido completo del análisis al diseño



Figura 8. Continuidad conceptual desde la necesidad hasta la interfaz.

El recorrido puede leerse de izquierda a derecha. Una necesidad detectada durante el relevamiento se formaliza mediante requerimientos. Los requerimientos se representan y analizan con UML. Los casos de uso ayudan a comprender las funciones; el diagrama de clases ayuda a comprender la estructura. Con esa base puede diseñarse una interfaz coherente y posteriormente desarrollarse la solución.

También puede recorrerse en sentido inverso para controlar trazabilidad. Si observamos una pantalla, deberíamos poder explicar qué caso de uso permite realizar y qué requerimiento justifica su existencia.

38. Qué debe dominar un alumno para rendir esta unidad

Un alumno que prepara la materia no debería limitarse a repetir definiciones. Debe demostrar comprensión y aplicación.

- Explicar por qué se modela antes de programar.
- Definir UML y diferenciarlo de una herramienta de dibujo o de un lenguaje de programación.
- Distinguir diagramas estructurales y de comportamiento.
- Elegir un tipo de diagrama según lo que necesita representar.
- Construir un Diagrama de Casos de Uso a partir de una problemática.
- Identificar actores, casos, asociaciones, include y extend.
- Explicar la dirección y el significado de las relaciones.
- Construir un Diagrama de Clases básico a partir de la misma problemática.
- Identificar clases, atributos, métodos, asociaciones, multiplicidades y herencia.
- Explicar encapsulamiento y la finalidad de la visibilidad.
- Diferenciar una funcionalidad de una clase.
- Relacionar requerimientos, casos de uso y clases mediante trazabilidad.
- Realizar los diagramas a mano o con una herramienta digital.
- Explicar cómo UML prepara el paso hacia el Diseño de Interfaz.

39. Actividades de estudio y comprobación

Las siguientes actividades permiten comprobar si el contenido fue comprendido. No deben resolverse solamente copiando definiciones: se espera aplicar los conceptos.

Actividad 1 - Conceptos

- Explique con sus palabras qué significa modelar un sistema y por qué es útil antes de programar.
- Explique qué es UML y por qué no debe confundirse con draw.io.
- Diferencie un diagrama estructural de uno de comportamiento y dé un ejemplo de cada uno.
- Explique la diferencia entre actor y persona concreta.
- Explique la diferencia entre <<include>> y <<extend>>.
- Explique la diferencia entre Caso de Uso y Clase.

Actividad 2 - Casos de Uso

Problemática: “Una biblioteca permite a sus socios buscar libros, reservar ejemplares disponibles y consultar sus préstamos. El bibliotecario registra libros y administra préstamos. Para reservar un ejemplar el sistema debe verificar previamente su disponibilidad.”

Identifique actores, casos de uso y relaciones. Luego realice el Diagrama de Casos de Uso a mano o en draw.io.

Actividad 3 - Clases

Utilizando la misma biblioteca, proponga las clases principales. Para cada clase indique al menos dos atributos y una operación. Establezca las relaciones y multiplicidades que pueda justificar a partir de la problemática.

Actividad 4 - Trazabilidad

Elija uno de los requerimientos de la biblioteca y construya una cadena: necesidad → requerimiento → caso de uso → clases relacionadas → posible interfaz.

Actividad 5 - Corrección de errores

Explique qué está mal en las siguientes decisiones: crear una clase llamada BuscarLibro solamente porque existe ese caso de uso; utilizar <<extend>> para una verificación que ocurre siempre; colocar un botón como caso de uso; inventar una multiplicidad sin conocer la regla del sistema.

40. Guía para estudiar antes de una evaluación

Primero debe comprenderse el recorrido general y no memorizar diagramas aislados. Conviene comenzar por explicar verbalmente qué problema se resuelve y qué requerimientos existen. Luego practicar la identificación de actores y objetivos, construir Casos de Uso y recién después analizar la estructura mediante clases.

Una buena forma de estudiar es tomar una problemática pequeña y realizar ambos diagramas sobre el mismo caso. Después se compara cada elemento con el texto original para justificar por qué existe.

Si un alumno puede mirar un diagrama y explicar qué significa cada figura, cada línea y cada relación, y además puede construir un modelo sencillo desde una consigna escrita, entonces no está solamente memorizando: está comprendiendo el contenido.

> Idea central de toda la unidad: UML no consiste en aprender a dibujar figuras. Consiste en aprender a representar de manera ordenada una solución de software. El dibujo es el resultado visible de un proceso previo de análisis.

Desarrollo de Sistemas - Modelado UML
Página 17